

HƯỚNG DẪN SINH VIÊN

TẠO AI AUDIT LOG

(RBL Insight Framework - AI Reflection 30%)

I. AI AUDIT LOG LÀ GÌ?

AI Audit Log KHÔNG phải là: Nơi copy-paste toàn bộ chat history với AI

AI Audit Log LÀ: Nhật ký ghi lại NHỮNG QUYẾT ĐỊNH QUAN TRỌNG khi làm việc với AI, chứng minh bạn có tư duy phản biện và tự chủ.

Mục đích:

- Chứng minh bạn hiểu code/giải pháp, không phải chỉ copy từ AI
- Thể hiện khả năng phát hiện lỗi sai của AI (hallucination)
- Chứng minh giá trị con người (Human Delta) mà AI không thể thay thế

II. CORE PROMPT LÀ GÌ?

Nguyên tắc vàng: Chỉ ghi những prompt mà nếu KHÔNG CÓ NÓ, project của bạn sẽ KHÁC VỀ BẢN CHẤT (kiến trúc, thiết kế, cách giải quyết), không phải chỉ khác về syntax hay formatting.

Ba loại Core Prompt (BẮT BUỘC ghi)

Loại Prompt	✅ CORE (Phải ghi)	❌ AUXILIARY (Không ghi)
Decision-Making	"Compare MVC vs Layered Architecture for this project"	"How to declare array in Java?"
Problem-Solving	"Why does my model overfit (train=0.95, test=0.62)?"	"Fix syntax error line 42"
Verification	"Is this paper Smith 2023 real?" → phát hiện fabrication	"Generate getter/setter for class"

💡 **Mẹo kiểm tra nhanh:** Nếu không có prompt này → Project vẫn giống 90% → LOẠI. Nếu không có prompt này → Project khác về bản chất → GHI.

III. SỐ LƯỢNG PROMPTS CẦN GHI

Loại bài	Core Prompts (Min-Max)	Hallucination Detection
----------	------------------------	-------------------------

Lab (PRF192)	8 - 12	≥ 1 case
Assignment (LAB211)	16 - 24	≥ 2 cases
Project (ADY201m)	15 - 20	≥ 3 cases
Project(DBI202)	5 - 8	≥ 2 cases
Project(PRO192)	10 - 16	≥ 2 cases

⚠ LƯU Ý: Mỗi DTC component (Decomposition, Pattern Recognition, Abstraction, Algorithms) phải có ÍT NHẤT 1 core prompt tương ứng.

IV. CÁCH ĐIỀN AI AUDIT LOG

Cấu trúc mỗi entry gồm 7 phần:

1. **Entry #:** Số thứ tự (001, 002, ...)
2. **Prompt Type:** DECISION / PROBLEM-SOLVING / VERIFICATION
3. **Stage/Component:** Decomposition, Pattern Recognition, Abstraction, Algorithms, hoặc Research stage
4. **Problem/Context:** Vấn đề đang gặp (1-2 câu ngắn gọn)
5. **Prompt to AI:** Câu lệnh gửi cho AI (NGUYỄN VĂN)
6. **AI Response (Summary):** Tóm tắt phản hồi của AI (2-4 câu)
7. **Human Delta & Reflection:** PHẦN QUAN TRỌNG NHẤT - Phải trả lời 4 câu hỏi:

4 câu hỏi bắt buộc trong Human Delta & Reflection:

- **Critical Thinking:** AI đúng/sai ở đâu? Phát hiện hallucination?
- **Contextualization:** Bối cảnh thực tế mà AI không biết?
- **Creative Synthesis:** Tôi đã thay đổi/kết hợp/tối ưu như thế nào?
- **Decision Ownership:** Quyết định cuối cùng và lý do?

V. VÍ DỤ CỤ THỂ

☒ Ví dụ TỐT (Entry hoàn chỉnh)

Entry #: 003

Prompt Type: DECISION-MAKING

Stage/Component: Algorithms

Problem/Context: Cần tìm kiếm player theo tên trong danh sách 10,000 records

Prompt to AI: "How to search for player by name in a list of 10,000 records efficiently?"

AI Response (Summary): AI suggested Binary Search with $O(\log n)$ complexity. Need to sort list first.

Human Delta & Reflection:

Critical Thinking: Binary Search đúng về lý thuyết NHƯNG cần sort trước $\rightarrow O(n \log n)$. Với search by partial name, Binary Search không phù hợp vì cần exact match.

Contextualization: Project yêu cầu 'partial match' (tìm 'Mess' $\rightarrow$ ra 'Messi'). Binary Search không support partial match.

Creative Synthesis: Tôi đã test 3 approaches: (1) Binary Search + exact match, (2) Linear Search + contains(), (3) HashMap + prefix search. Kết quả: HashMap fastest (2ms) vs Linear (15ms) vs Binary (12ms + sort time).

Decision Ownership: Chọn HashMap vì: faster lookup $O(1)$, support partial match với custom comparator, trade-off: tốn memory hơn nhưng acceptable với 10K records.

Evidence: Screenshot comparison table, code snippet HashMap implementation

✗ Ví dụ KÉM (Không đạt)

Entry #: 007

Human Delta & Reflection:

"AI helped me implement the search function. The code works well."

Tại sao KÉM:

- Không trả lời 4 câu hỏi
- Không có critical thinking (AI đúng hay sai?)
- Không có decision-making process
- Không có evidence

VI. PHÁT HIỆN HALLUCINATION (BẮT BUỘC)

Hallucination là gì? Khi AI đưa ra thông tin SAI nhưng trình bày rất tự tin (fake papers, fake APIs, logic errors)

5 loại Hallucination phổ biến:

8. **Fabrication:** AI tạo ra papers, APIs không tồn tại
9. **Oversimplification:** AI bỏ qua edge cases, trường hợp ngoại lệ
10. **Logic Error:** AI nói 'XGBoost always best' $\rightarrow$ Sai, phụ thuộc dataset
11. **Outdated Info:** AI dùng thông tin cũ, không còn đúng
12. **Context Misunderstanding:** AI không hiểu đúng bối cảnh business/technical

Ví dụ Hallucination Detection:

Prompt: "Summarize 5 recent papers on IoT anomaly detection"

AI Response: Listed 5 papers including 'Smith et al. (2023)'

Detection: Searched Google Scholar $\rightarrow$ Paper NOT FOUND $\rightarrow$ FABRICATION

Corrective Action: Found real paper 'Jones et al. (2022)' and used instead

VII. CHECKLIST TỰ KIỂM TRA (TRƯỚC KHI NỘP)

Kiểm tra **MỖI ENTRY** phải pass $\geq 4/5$ tiêu chí:

- ☐ Prompt này ảnh hưởng đến quyết định quan trọng trong project?
- ☐ Nếu không có prompt này, project có thay đổi về architecture/design/approach?
- ☐ Tôi có thể giải thích lý do tại sao chọn/không chọn gợi ý của AI?
- ☐ Có minh chứng cụ thể (code, metrics, comparison) cho quyết định này?
- ☐ Prompt này phản ánh tư duy phản biện, không phải chỉ copy AI output?

⚠ Nếu entry KHÔNG pass $\geq 4/5$ → LOẠI BỎ, không ghi vào Log

Kiểm tra **TỔNG THỂ Log**:

- ☐ Số lượng entries nằm trong range (min-max) theo loại bài?
- ☐ Mỗi DTC component có ít nhất 1 core prompt?
- ☐ Đã phát hiện $\geq$ số lượng hallucination yêu cầu?
- ☐ Mỗi entry đều có Human Delta đầy đủ (4 câu hỏi)?
- ☐ Có evidence cho $\geq 70\%$ entries?

VIII. LƯU Ý QUAN TRỌNG

- **AI Audit Log chiếm 30% tổng điểm - KHÔNG THỂ BỎ QUA**
- Giảng viên sẽ hỏi vấn đáp (Oral Vivas) về Log - Phải trả lời được
- Log giả/kém chất lượng → 0 điểm AI Reflection (mất 30%)
- Chất lượng quan trọng hơn số lượng - Đừng ghi nhiều entries trivial
- Nộp đúng deadline - Log là bộ phận **BẮT BUỘC** của bài nộp

Chúc các bạn hoàn thành AI Audit Log thành công!

© 2026 FPT University - Computing Fundamental Department